Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №54

ТЕМА: Створення сучасної віконної програми з темним інтерфейсом (наприклад, шифрувальник або обробник тексту).

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- закріпити всі здобуті навички роботи з CustomTkinter в одному завершеному проєкті;
- навчитися поєднувати введення даних, обробку та виведення результату в єдиному стилізованому вікні;
- розвинути навички проєктування інтерфейсу «від задачі», а не «від віджетів»;
- отримати практичний досвід створення сучасної, візуально цілісної віконної програми.

Теоретичні відомості

1. Від окремих віджетів до цілісного застосунку

На попередніх заняттях ми детально розглянули кожен віджет CustomTkinter окремо. Створення реального застосунку — це насамперед про те, як грамотно скомпонувати вже відомі елементи в єдиний, логічно послідовний і візуально цілісний інтерфейс, що розв'язує конкретну практичну задачу. Гарний інтерфейс керується не «якими віджетами я хочу скористатися», а «яку послідовність дій має пройти користувач, щоб отримати потрібний результат» — і вже під цю послідовність підбираються відповідні віджети.

2. Проєктування інтерфейсу «шифрувальника тексту»

Розглянемо повний приклад проєкту — застосунок для шифрування тексту шифром Цезаря (уже знайомим із теми 3). Розіб'ємо задачу на логічні кроки, які й визначать структуру інтерфейсу: 1) користувач вводить текст повідомлення; 2) користувач вводить ключ (величину зсуву); 3) користувач обирає дію — зашифрувати чи розшифрувати; 4) програма показує результат.

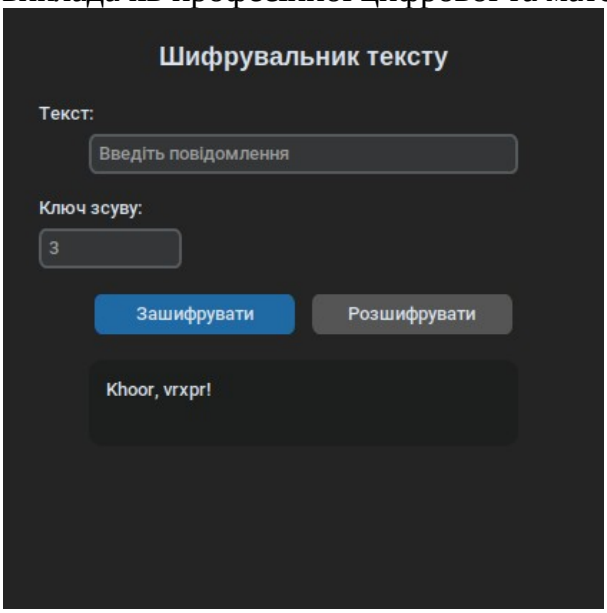


Рис. 1. Готовий інтерфейс застосунку-шифрувальника: поля введення, дві кнопки дій і поле результату

3. Реалізація логіки шифрування

Функцію шифрування, розроблену в темі 3, можна використати без жодних змін — GUI лише викликає її з параметрами, отриманими з полів уведення, замість input():

```
def caesar_cipher(text, key):
    result = ''
    for ch in text.upper():
        if ch.isalpha():
            shifted = (ord(ch) - ord('A') + key) % 26
            result += chr(ord('A') + shifted)
        else:
            result += ch
    return result
```

Такий поділ програми на «логіку» (звичайні функції, що не залежать від інтерфейсу) і «інтерфейс» (віджети та їхні обробники) є хорошою практикою: функцію caesar_cipher() можна перевірити окремо, без запуску вікна, а за потреби — використати повторно і в іншій, консольній чи вебверсії програми.

4. Обробники кнопок дій

Обидві кнопки («Зашифрувати» і «Розшифрувати») викликають ту саму функцію caesar_cipher(), відрізняючись лише знаком ключа зсуву (для розшифрування ключ береться з протилежним знаком):

```
def on_encrypt():
    try:
        text = text_entry.get()
        key = int(key_entry.get())
```

```
        result = caesar_cipher(text, key)
        result_box.delete('0.0', 'end')
        result_box.insert('0.0', result)
    except ValueError:
        result_box.delete('0.0', 'end')
        result_box.insert('0.0', 'Помилка: ключ має бути цілим
числом')

def on_decrypt():
    try:
        text = text_entry.get()
        key = int(key_entry.get())
        result = caesar_cipher(text, -key)
        result_box.delete('0.0', 'end')
        result_box.insert('0.0', result)
    except ValueError:
        result_box.delete('0.0', 'end')
        result_box.insert('0.0', 'Помилка: ключ має бути цілим
числом')
```

Метод delete('0.0', 'end') очищає поле результату перед виведенням нового значення — без цього кроку новий результат просто дописувався б до попереднього, і текстове поле поступово заповнювалося б історією всіх обчислень.

5. Композиція повного вікна

Останній крок — зібрати всі раніше описані частини (мітки, поля введення, кнопки, поле результату) в одному вікні, дотримуючись єдиної стилістики (однакові відступи, узгоджені кольори кнопок, помірне використання жирного шрифту лише для заголовка):

```
import customtkinter as ctk

ctk.set_appearance_mode('Dark')
ctk.set_default_color_theme('blue')
app = ctk.CTk()
app.title('Шифрувальник тексту')
app.geometry('420x420')

ctk.CTkLabel(app, text='Шифрувальник тексту', font=('Arial', 18,
'bold')).pack(pady=(20, 12))
ctk.CTkLabel(app, text='Текст:').pack(anchor='w', padx=25)
text_entry = ctk.CTkEntry(app, width=300)
text_entry.pack(padx=25, pady=(0, 10))
ctk.CTkLabel(app, text='Ключ зсуву:').pack(anchor='w', padx=25)
key_entry = ctk.CTkEntry(app, width=100)
key_entry.pack(padx=25, pady=(0, 10), anchor='w')

btn_row = ctk.CTkFrame(app, fg_color='transparent')
btn_row.pack(pady=8)
ctk.CTkButton(btn_row, text='Зашифрувати',
command=on_encrypt).pack(side='left', padx=6)
```

```
ctk.CTkButton(btn_row, text='Розшифрувати', command=on_decrypt,
fg_color='#555555').pack(side='left', padx=6)

result_box = ctk.CTkTextbox(app, width=300, height=60)
result_box.pack(pady=10)

app.mainloop()
```

Зверніть увагу на порядок визначення функцій і віджетів: функції on_encrypt() і on_decrypt() звертаються до text_entry, key_entry і result_box, тому в остаточному файлі ці функції-обробники прийнято розміщувати після оголошення відповідних віджетів (або, як у деяких стилях коду, всі функції — на початку файлу, а віджети створювати вже після них, якщо використовувати глобальні змінні обережно й послідовно).

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Ключ у завданні 1 і колір кнопок залежать від N (mod — остача від ділення):

Завдання	**Дані варіанта**	**Приклад N = 7**
1, 3	ключ k = 3 + N mod 20	k = 10
5	fg_color за N mod 3 = 0, 1, 2: #1f6aa5, #2e8b57, #8e44ad	#2e8b57

б) Кожне завдання — окремий файл Прізвище_ПР54_зN.py (наприклад, Ivanenko_PR54_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного завдання додайте скріншот вікна із видимим заголовком. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів і повідомлень — точно як в умові. Логіку (функції шифрування, аналізу) відокремте від віджетів і викликайте з обробників.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР54_з1.py. Реалізуйте функцію caesar_cipher(text, shift) окремо від інтерфейсу (великі й малі латинські літери зсуваються, інші символи не змінюються) і перевірте викликом print() без GUI.

   Приклад (N = 7): caesar_cipher('HELLO', 3) → KHOOR; caesar_cipher('HELLO', 10) → ROVVY; caesar_cipher('Hello, World!', 3) → Khoor, Zruog!.

2. Побудуйте інтерфейс: заголовок, поле тексту, поле ключа, кнопки «Зашифрувати» і «Розшифрувати», поле результату (CTkTextbox).

   Приклад: усі елементи видно у вікні 500×450 у темному оформленні.

3. Реалізуйте обробники кнопок; для нецілого ключа виведіть у полі результату «Ключ має бути цілим числом».

   Приклад (N = 7): текст «HELLO», ключ 10 → «ROVVY»; розшифрування «ROVVY» з ключем 10 → «HELLO»; ключ «x» → «Ключ має бути цілим числом».

4. Додайте кнопку «Очистити», що очищає поля тексту, ключа й результату.

   Приклад: після натискання всі три поля порожні.

5. Стилізуйте застосунок: fg_color кнопок вашого варіанта, hover_color — темніший відтінок, corner_radius = 12 для всіх елементів.

   Приклад (N = 7): fg_color = «#2e8b57».

6. Завдання підвищеної складності*. Додайте кнопку «Аналіз», що виводить у полі результату «Літер: …», «Слів: …», «Найчастіша літера: …» (регістр не враховується; за однакової частоти — перша за алфавітом).

   Приклад: «Hello world» → «Літер: 10», «Слів: 2», «Найчастіша літера: l».

Критерії оцінювання

Складова	**Оцінювання**	**Макс. балів**
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, логіка змішана з віджетами,

не очищується поле результату перед новим виводом, не оброблено нечисловий ключ, змінюються символи, що не є літерами) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, додати нову дію чи змінити параметри шифру). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Чому корисно відокремлювати «логіку» (функції обчислень) від «інтерфейсу» (віджети й обробники подій)?
2. Як в одному застосунку зв'язати кілька кнопок з однією й тією самою логічною функцією, які відрізняються лише параметром?
3. Для чого перед виведенням нового результату в CTkTextbox спочатку викликають delete('0.0', 'end')?
4. Які кроки, на вашу думку, доцільно виконати перш ніж починати писати код, проєктуючи новий GUI-застосунок?